

**Facial Segmentation in Images:**  
**Separation and Thresholding of Face and Eyes**

Noah Hester and Christian Mason and Michael Gilman

COMP/MATH/PHYS 345: Digital Image Processing

Dr. William Barber

November 27, 2024

## **Abstract**

In this project, we attempt to create automated preliminary processes for facial recognition which apply methods learned across the class to adequately segment human faces in a given image. We apply these methods in three main stages consisting of preprocessing, elliptical fitting, and masking, first attempting to segment simple ID photos, then moving to slightly more complicated images. Preprocessing involves filtering and edge detection for the elliptical fitting, which generates regions of interest. Masking takes the regions of interest and attempts to completely isolate the face from the background or other objects. We found that these techniques are solid in their own right, but best results would likely be achieved with more complex machine learning methods.

## **Introduction**

In this project, we attempt to create automated preliminary processes for facial recognition which apply methods learned across the class to adequately segment human faces in a given image. The three primary areas of focus are Preparation of the image, Elliptical detection, and Mask creation. We attempt to combine these three processes and to refine each step in order to understand and streamline the automation, and to begin the needed set up for a future facial recognition system that will be able to handle the widest set of cases. Each step will be covered in this paper and their respective limitations will be highlighted.

## **Methods**

### **Operating Systems:**

- MacOS Sequoia (Version 15.1.1)
- Windows 11 (Version 10.0.22631)

### **Hardware Specifications:**

- 2023 Macbook Air
  - Processor: Apple M2
  - RAM: 8 GB
- 2022 HP Pavilion Gaming Laptop
  - Processor: Intel Core i5-10300H
  - GPU: NVIDIA GTX
  - RAM: 8 GB

### **Software Tools and Versions:**

- Code Editor:
  - Visual Studio Code (Version 1.95.3)
- Programming Languages:
  - Python (Version 3.11.8)

### **Dependencies and Libraries:**

- Numpy (Version 1.26.4)
- OpenCV (Version 4.9.0.80)
- Scikit-image (Version 0.20.0)

For this project, a number of general methods were used in order to achieve our final results. Before any attempts at face detection of any sort, the image is sent through a stage of **preprocessing**. This gets the image ready to be passed on to the next method, which is that of **elliptical detection/fitting**. This splits the image into relevant segments that are individually processed in the **masking** step. These are then fitted back together into a full sized mask that can be used on the original image to isolate the faces from the rest of the image.

**Preprocessing** of an image is required to introduce a clean image to the elliptical detection step. First an image is converted to an eight bit gray scale and in the finished system this initial step will also resize each image in order to create uniformity within the code. The image is smoothed by convolution with a median or gaussian kernel. The image is then contrast adjusted (in our case a simple normalization) in order to create stronger gradients for the edge detection. A canny edge detection is applied and contingent on the images' needs, the gaussian convolution step of the canny must be tuned. It is also highly dependent on the thresholding arguments of the Python OpenCV Canny function, which currently are not automated within our system and must be manually adjusted to detect the images' meaningful gradients. Once a clean edge detection is produced we can pass this image to the next step.

**Elliptical detection** consists of the Python OpenCV find contours function as well as the OpenCV contour area and fit ellipse functions. First, the contours are identified in the edge detected image with a raster scan that will consider neighboring pixels and help to identify the elliptical shape. After this step, arguments are provided for various logistical thresholdings (ie. Size, Shape, and Area). These are manually set in our project and will require a more in depth system to functionally automate. When the OpenCV fit ellipse function is guided correctly it will identify ellipticals from partial arc lengths. The goal is to draw the ellipticals about each face and create a region of interest for a cropped image, which we then send down the line for masking.

**Masking** is the third and final step of our facial segmentation process. It takes in a cropped image and attempts to subtract the background from the image to get a clearer isolation of the subject. If this is unsuccessful, then this step is skipped and we move straight to the next one, which is applying a Gaussian blur. This will help smooth the boundaries of the final mask when thresholding is performed. After this, some contrast enhancement may be performed to

help isolate the objects of interest from the background. For thresholding, it is much more human-dependent, so to speak, since the varying types of background objects and lighting conditions on the face require different thresholding values for best isolation. Different thresholding methods were tried, and manual adjustment works the best to specify for each image, but other attempted methods include Otsu thresholding and thresholding based on histogram minima values. Once the majority of the face has been isolated from the background in a closed shape, we can perform a particle/contour analysis on the image. In ImageJ, we perform a particle analysis and filter based on the size of the particle and roundness to isolate the face mask portions. In Python, we can use the OpenCV `findContours()` function on the binary image to get the contours and then use that to find areas to compare. Once we have the pieces of face, we can then attempt to fill in some of the breaks or holes in the mask by making the mask pieces black on a white background and performing the watershed function. This will create lines that will help connect pieces of the face mask to each other, allowing us to invert it and fill holes to create a more complete face mask. Following this, a median filter of radius 1 is then applied in order to remove any leftover watershed lines. If the mask is a bit too small or large, we can apply some dilation or erosion to help fix the sizing. Once we are satisfied with the face mask, we position it back into a larger mask in the same place that the original chunk came out of the original image. This allows us to construct a single mask for the entire image composed of the individual masks from the regions of interest.

This method did not entirely transfer to a Python program as it required too much human interaction. In the hopes of reducing this, a different method was developed. First, a rolling ball background subtraction is applied with a radius of 15, then similar preprocessing techniques as before with an Otsu thresholding applied afterwards. A contour analysis is then performed on

this result to analyze the particles, and the largest particle is selected in the goal of isolating the main component with the face. An erosion of radius 2 is then applied with 5 iterations to reduce the border around the face from the background subtraction and to separate the face from any other artifacts included in its contour. This is then passed through the particle analysis again in order to isolate the face, and then this is taken as the final mask.

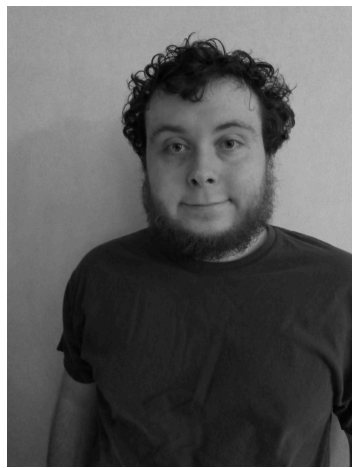
## Results

The results will be displayed categorically according to the three general steps of our project.

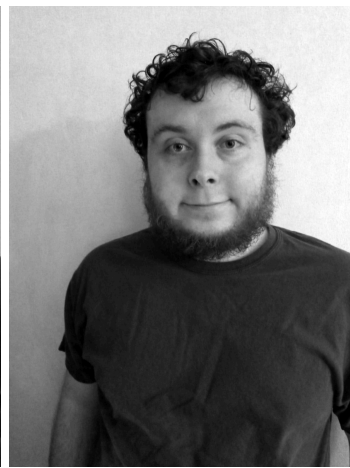
**Preprocessing** results are strong, given that the class focused on developing these tools.



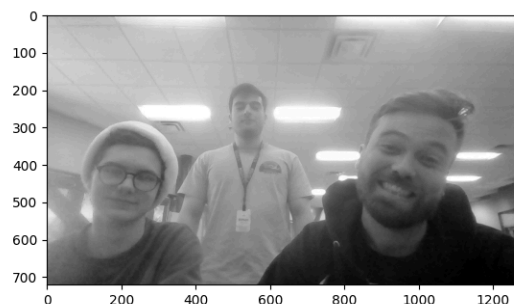
Original



Grayscale



Normalising



And here we see an example of severe glare and lighting disparities, which in our preprocessing we were not able to handle and led to bad results.

**Elliptical detection** was performed accurately at times but required heavy moderation.



Result of faulty elliptical detection. An abundance of errors related to edge detection/elliptical detection arguments.



Result of edge detection suffering from lighting conditions and bad gradient thresholding.



Exemplifications of both strong elliptical detection arguments and preprocessing and weak preprocessing with poor arguments.



Images which show the logic in the methodology. Strongly parsed regions of interest lead to streamline personalised mask creation, as each mask becomes independent of other faces and features.

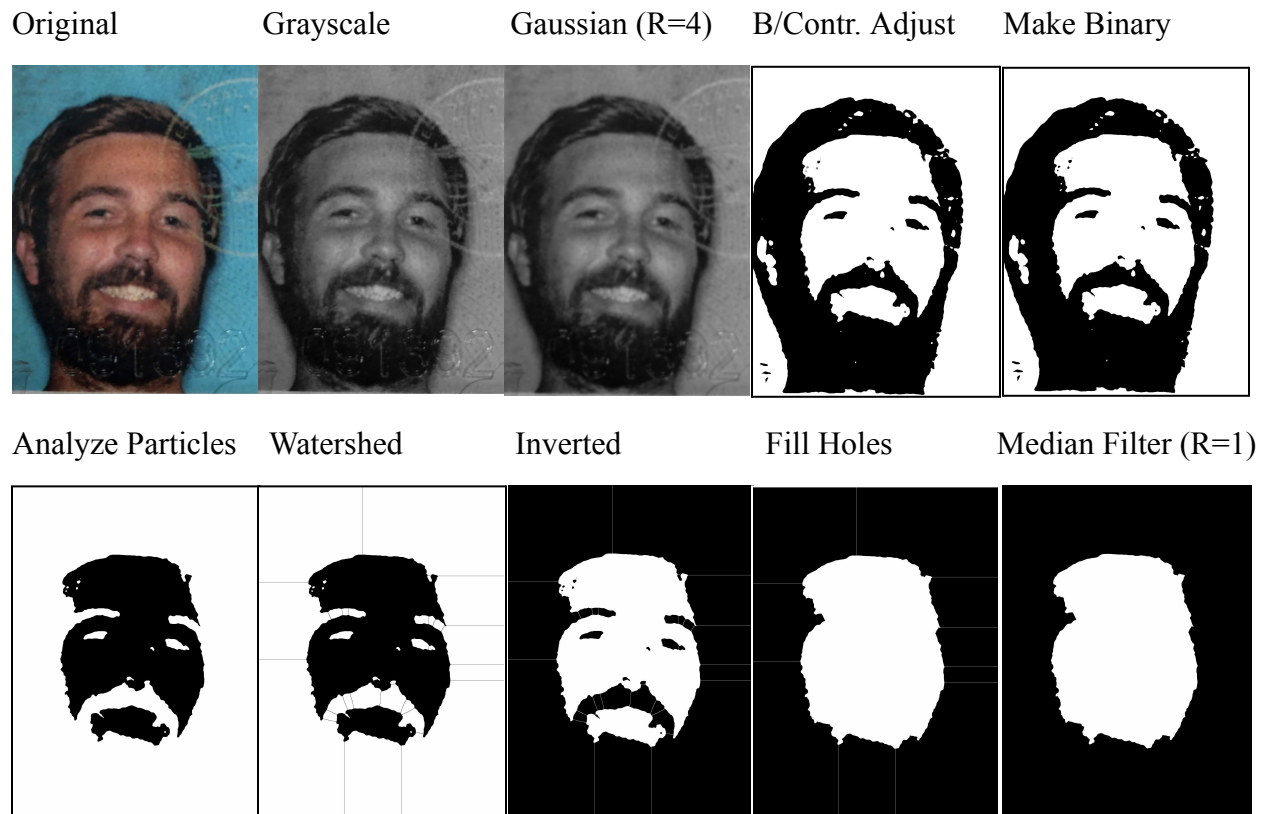


**Masking** was successful in some cases, but required a lot of manual adjustment to achieve the best results from the process.

These are three successful segmentations performed manually in ImageJ.

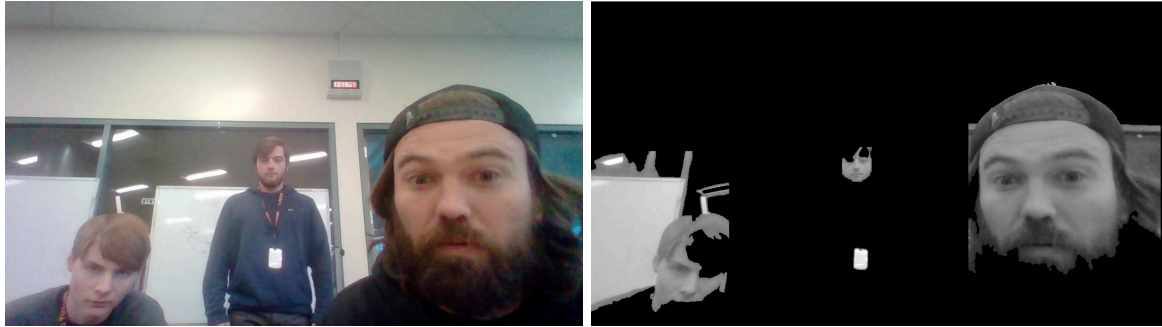


They were created using the following process as previously described:



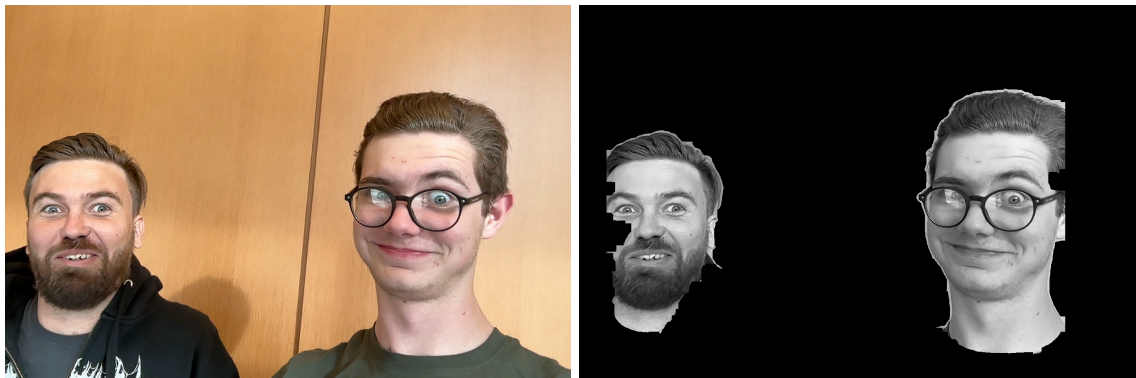
The final program to incorporate all of these techniques gave promising results after some tweaking of different methods. This was the result of a previous method of masking in the

program using some preprocessing techniques like histogram equalization and median, blurring, and maximum filters, with a thresholding algorithm that attempted to analyze minima in the histogram and find ideal minima to threshold on.



As we can see, it partially works, but is not entirely ideal. To help fix this, a differing method was devised, which is the one described at the end of the method.

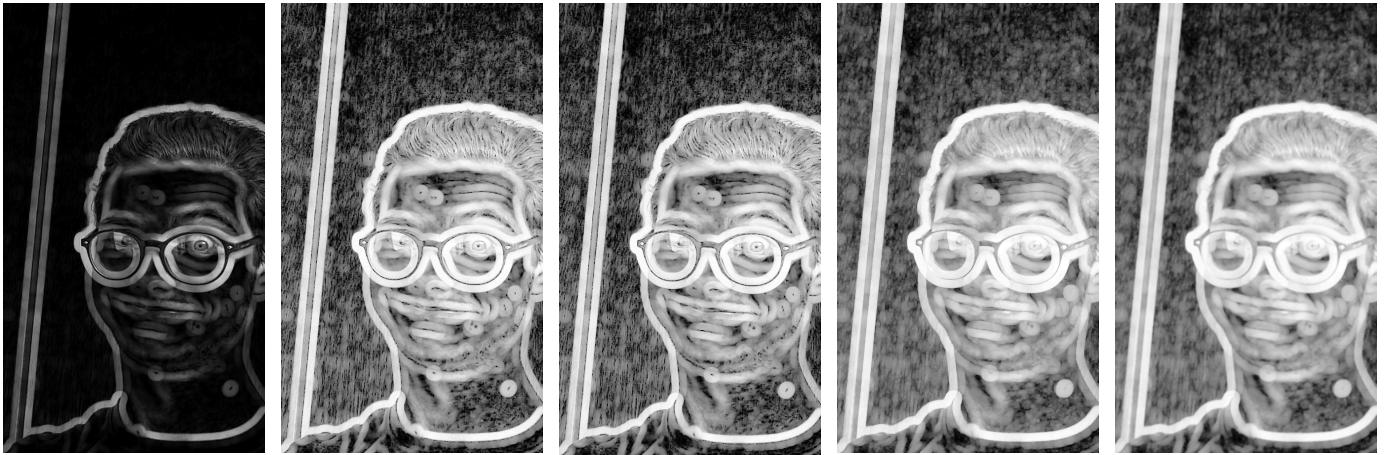
These are the results of the alternative method:



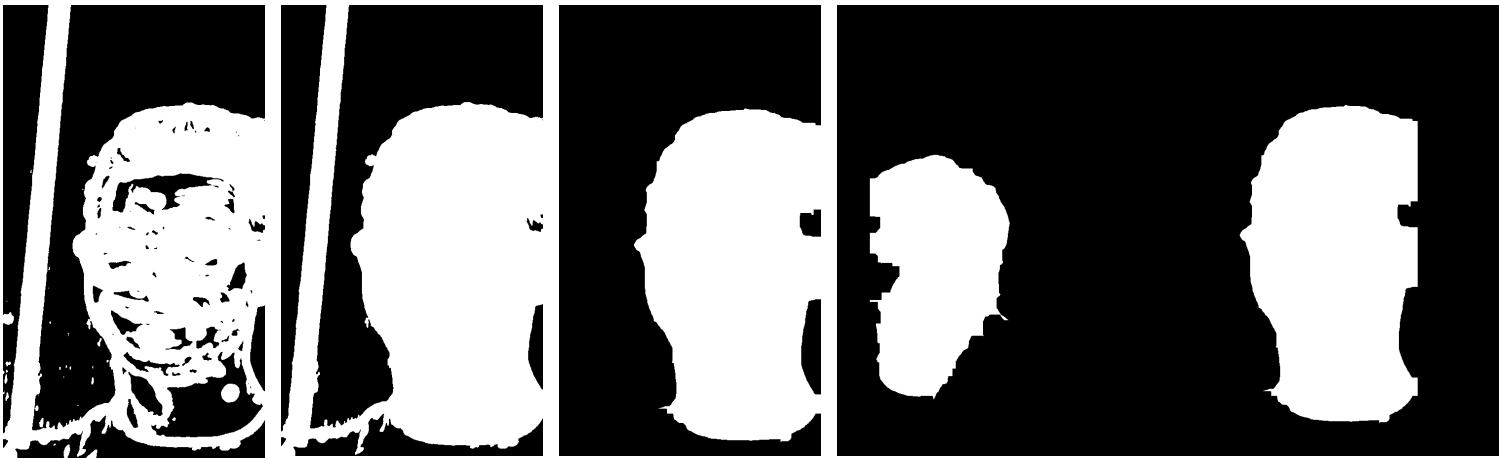
Original

Result of masking program

These are the intermediate steps in the process (displayed on a single face from the splitting step)



|                       |           |        |         |               |
|-----------------------|-----------|--------|---------|---------------|
| Subtracted Background | Equalized | Median | Maximum | Gaussian Blur |
|-----------------------|-----------|--------|---------|---------------|



|                   |                 |                                   |                                       |
|-------------------|-----------------|-----------------------------------|---------------------------------------|
| Otsu Thresholding | Largest Contour | Erosion and Largest Contour Again | Placement back into larger final mask |
|-------------------|-----------------|-----------------------------------|---------------------------------------|

Final Result:



## **Conclusion**

We have ultimately found that the processes are sound in theory but require more intricate systems in practice. The results clearly show a possibility to segment via logical avenues as well as algorithmic and mathematical methods to create relatively accurate masks. In order for our facial recognition system to work as intended, the preprocessing techniques must be slightly adjusted but remain similar given an arbitrary image. The elliptical detection methods are crude and require a small overhaul with less stringent constraints and more layers of logical thresholding. The mask generation itself is somewhat successful, but requires heavy monitoring to get the best results. In order to fully automate and generalize the process, machine learning methods can be implemented in the future.